

Master Architecture Plan: Live-Sync Memory Configuration System

Version: 1.0

Date: June 12, 2026

Author: Gaurav Kumar

Organization: Gaurav Kumar

Table of Contents

1. Executive Summary
2. Core Mathematical Engine ($T / R / C$)
3. System Presets Matrix
4. Backend Schema Synchronization
5. Frontend UI Layout Strategy
6. State Synchronization and Interactive Data Flow
7. Implementation Roadmap
8. Appendices

Executive Summary

Purpose

This document outlines the **architecture, implementation, and workflow** for the **Live-Sync Memory Configuration System**. The system is designed to efficiently manage context tokens in a sliding-window architecture, ensuring optimal performance, cost-effectiveness, and user control.

Key Components

- **Core Mathematical Engine:** Governs the sliding-window architecture using the formula $C = T - R$.
- **System Presets Matrix:** Predefined configurations for different use cases (Precise, Balanced, Turbo, Custom).
- **Backend Schema Synchronization:** Dynamic ingestion of configuration states to replace hardcoded values.
- **Frontend UI Layout:** User-friendly interface for configuring memory settings.
- **State Synchronization:** Seamless data flow between frontend and backend.

Expected Outcomes

- Efficient context token management.
- Dynamic backend-frontend synchronization.
- User-friendly configuration UI with preset and custom modes.
- Robust validation to prevent runtime errors.

Core Mathematical Engine (\$T / R / C\$)

Formula & Variables

The system relies on three primary variables to manage context tokens:

- **Trigger Threshold (\$T\$):** Maximum number of messages allowed in the active chat window before compression is forced.
- **Raw Buffer (\$R\$):** Number of recent messages preserved intact at the end of the history.
- **Calculated Chunk Size (\$C\$):** Block of older messages between the system prompt and the raw buffer, removed and fused into a rolling summary.

Formula: $C = T - R$

Guardrail Constraints

To prevent runtime errors, the following constraints must be enforced:

1. **Dominance Rule:** $T > R$ (Ensures $C > 0$).
2. **Processing Floor:** $T - R \geq 5$ (Prevents expensive LLM calls for tiny chunks).
3. **Safe Lower Bounds:** $T \geq 10$ and $R \geq 2$ (Prevents timeout loops).

Action Items

Task	Description	Responsible Party	Deadline
Validate mathematical constraints	Ensure T , R , and C adhere to guardrails.	Backend Team	June 19, 2026
Implement formula in backend	Integrate $C = T - R$ into compression logic.	Backend Team	June 22, 2026
Test edge cases	Verify behavior when $T \leq R$ or $C < 5$.	QA Team	June 25, 2026

System Presets Matrix

Preset Profiles

The system offers four preset configurations:

Profile	Trigger Threshold (\$T\$)	Raw Buffer (\$R\$)	Calculated Chunk (\$C\$)	Summary Token Cap	Grounding Pass Interval	Optimization Target
Precise	40 msg	15 msg	25 msg	1200 tkn	Every 3 epochs	Deep analytical sessions
Balanced	30 msg	10 msg	20 msg	800 tkn	Every 5 epochs	Standard performance
Turbo	15 msg	4 msg	11 msg	500 tkn	Every 7 epochs	Extreme speed, cost minimization
Custom	User Defined	User Defined	Auto-Computed	User Defined	User Defined	Granular control

Action Items

Task	Description	Responsible Party	Deadline
Define preset configurations	Hardcode preset values in backend.	Backend Team	June 20, 2026
Create UI for preset selection	Implement 3-step slider in Configuration.jsx.	Frontend Team	June 23, 2026
Test preset switching	Ensure seamless transitions between presets.	QA Team	June 26, 2026

Backend Schema Synchronization

Incoming POST Payload Structure

The backend will dynamically ingest the configuration state via a POST payload:

```
{
  "memory_preset": "balanced",
  "memory_trigger_threshold": 30,
  "memory_grounding_interval": 5
}
```

Schema Updates

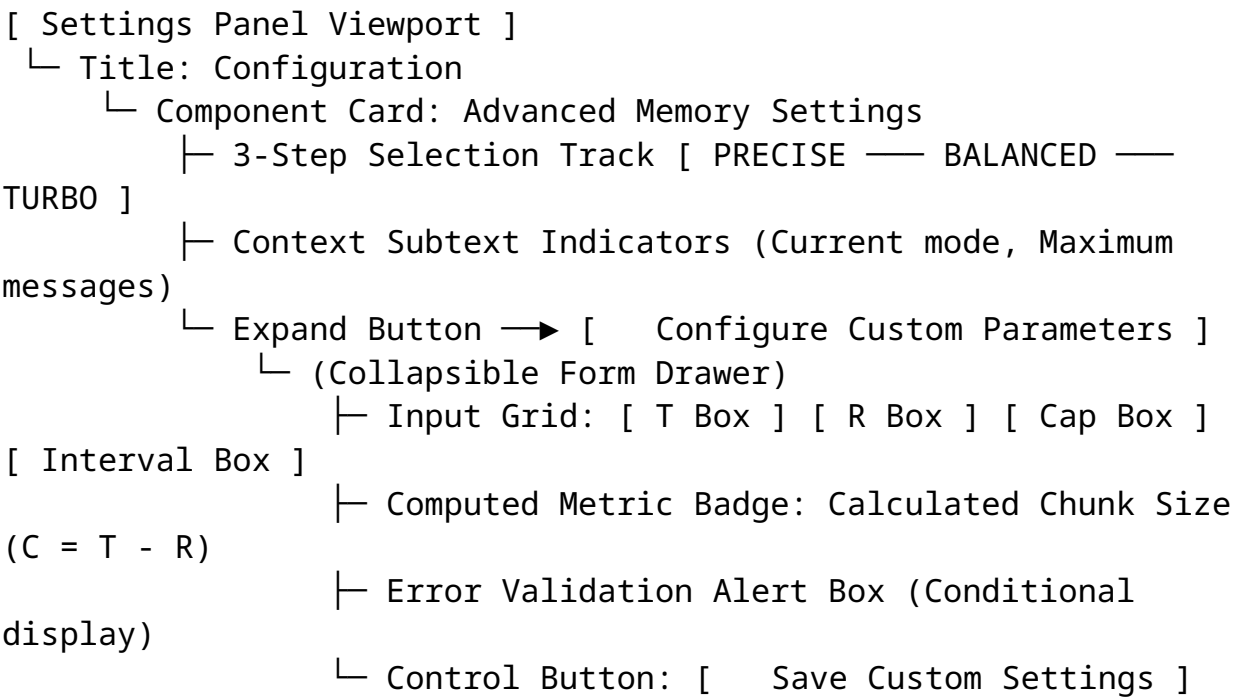
- **File:** backend/routes/chat.py
- **Changes:**
 - Add memory_preset: Optional[str] = "balanced" to ChatRequest.
 - Add memory_grounding_interval: Optional[int] = 5 to ChatRequest.
 - Replace hardcoded 5 in stream_chat_response with req.memory_grounding_interval.

Action Items

Task	Description	Responsible Party	Deadline
Update Pydantic schema	Add new fields to ChatRequest.	Backend Team	June 21, 2026
Refactor grounding check	Replace hardcoded interval with dynamic value.	Backend Team	June 22, 2026
Test payload validation	Ensure backend accepts and processes new fields.	QA Team	June 24, 2026

Frontend UI Layout Strategy

Component Hierarchy



UI Component States

- **3-Step Slider:** Auto-saves preset on change.
- **Custom Drawer:** Collapsible form for manual overrides.
- **Validation Alerts:** Crimson warnings for guardrail violations.

Action Items

Task	Description	Responsible Party	Deadline
Design UI layout	Create mockups for <code>Configuration.jsx</code> .	UI/UX Team	June 20, 2026
Implement slider component	Develop 3-step range selector.	Frontend Team	June 23, 2026
Add validation logic	Display errors for invalid <code>\$T\$</code> , <code>\$R\$</code> , or <code>\$C\$</code> .	Frontend Team	June 24, 2026
Test UI responsiveness	Ensure compatibility across devices.	QA Team	June 27, 2026

State Synchronization and Interactive Data Flow

User Actions & Flows

- Flow A: Slider Interaction (Preset Auto-Save Engine)**
 - User drags slider → Updates local state → Syncs to `sessionStorage`.
- Flow B: Manual Overrides (Explicit Custom Save Engine)**
 - User edits inputs → Validates constraints → Saves to `sessionStorage` → Sets preset to `'custom'`.
- Flow C: Runtime Chat Delivery Pipeline**
 - Fetches config from `sessionStorage` → Injects into `POST /api/chat` payload.

Action Items

Task	Description	Responsible Party	Deadline
Implement <code>sessionStorage</code> sync	Save/load config from browser storage.	Frontend Team	June 25, 2026
Integrate config into chat payload	Inject config into <code>/api/chat</code> requests.	Backend Team	June 26, 2026
Test end-to-end flow	Verify data consistency from UI to backend.	QA Team	June 28, 2026

Implementation Roadmap

Phase 1: Backend Setup (June 19–22, 2026)

- Validate mathematical constraints.
- Update Pydantic schema.
- Refactor grounding check.

Phase 2: Frontend Development (June 20–25, 2026)

- Design UI mockups.
- Implement slider and custom drawer.
- Add validation logic.

Phase 3: Integration & Testing (June 26–28, 2026)

- Sync sessionStorage with backend.
 - Test preset switching and manual overrides.
 - End-to-end validation.
-

Appendices

Appendix A: Glossary of Terms

- **\$T\$ (Trigger Threshold)**: Maximum messages before compression.
- **\$R\$ (Raw Buffer)**: Recent messages preserved.
- **\$C\$ (Calculated Chunk Size)**: Older messages for compression.
- **Guardrail Constraints**: Rules to prevent runtime errors.
- **Preset Profiles**: Predefined configurations (Precise, Balanced, Turbo, Custom).

Appendix B: Sample Code Snippets

Backend Schema Updates

```
# backend/routes/chat.py
from pydantic import BaseModel, Optional

class ChatRequest(BaseModel):
    memory_preset: Optional[str] = "balanced"
    memory_trigger_threshold: Optional[int] = 30
    memory_grounding_interval: Optional[int] = 5
```

Frontend Slider Component

```
// Configuration.jsx
const presets = [
  { name: "Precise", T: 40, R: 15, C: 25, cap: 1200, interval:
3 },
  { name: "Balanced", T: 30, R: 10, C: 20, cap: 800, interval:
5 },
  { name: "Turbo", T: 15, R: 4, C: 11, cap: 500, interval: 7 },
];

const handleSliderChange = (index) => {
  const preset = presets[index];
  setConfig(preset);
  sessionStorage.setItem('agent_memory_config',
JSON.stringify(preset));
};
```

Appendix C: Testing Checklist

☐

Validate $T > R$ and $T - R \geq 5$.

☐

Test preset switching (Precise, Balanced, Turbo).

☐

Verify custom mode saves and locks slider.

☐

Ensure sessionStorage syncs with backend.

☐

Test edge cases (e.g., $T = R$, $C < 5$).

References

- Pydantic Documentation
- React.js Documentation
- Internal backend/frontend framework docs.